

KislistScraper_Pro_New — pełny projekt Scrapy

Poniżej znajdziesz kompletny projekt **Scrapy** przygotowany jako szkielet, który implementuje to, o czym rozmawialiśmy: per-domena spiders dla Domni, Łazienkarium, Dekordia, pipeline do zapisywania obrazów w docelowej strukturze oraz pomocne skrypty uruchomieniowe.

Uwaga: To jest kompletny projekt zapisany w jednym pliku markdown. Skopiuj poszczególne pliki i zapisz je w strukturze projektu dokładnie tak, jak niżej.

Struktura projektu (co zapisać)

```
KislistScraper_Pro_New/  
├─ scrapy.cfg  
├─ requirements.txt  
├─ run_all.sh  
├─ KislistGUI.desktop  
├─ README.md  
└─ kislist_scraper/  
    ├─ __init__.py  
    ├─ items.py  
    ├─ middlewares.py  
    ├─ pipelines.py  
    ├─ settings.py  
    └─ spiders/  
        ├─ __init__.py  
        ├─ base_spider.py  
        ├─ domni_spider.py  
        ├─ lazienkarium_spider.py  
        └─ dekordia_spider.py
```

requirements.txt

```
Scrapy>=2.8  
requests>=2.28  
beautifulsoup4>=4.12  
lxml>=4.9  
Pillow>=9.0
```

scrapy.cfg

```
[settings]
default = kislist_scraper.settings
```

run_all.sh

```
#!/usr/bin/env bash
set -e
PROJECT_DIR="$(cd "$(dirname "$0")" && pwd)"
cd "$PROJECT_DIR"
# Example usage: run three test spiders – pass URLs as args (or edit below)
# You can edit URLs or call each spider separately.

# Example: crawl single product pages – replace with real URLs or wrap in
loop
# scrapy crawl dekordia -a url="https://dekordia.pl/produkt/...."
# scrapy crawl domni -a url="https://domni.pl/produkt/..."
# scrapy crawl lazienkarium -a url="https://lazienkarium.pl/p/...."

# For convenience: open interactive prompt if no args
if [ "$#" -eq 0 ]; then
    echo "No args provided. Example commands:"
    echo "  scrapy crawl dekordia -a url=URL"
    echo "  scrapy crawl domni -a url=URL"
    echo "  scrapy crawl lazienkarium -a url=URL"
    exit 0
fi

# If args provided, forward to scrapy (first arg should be spider name)
# Example: ./run_all.sh dekordia "https://..."
spider="$1"
shift
scrapy crawl "$spider" "$@"
```

Make `chmod +x run_all.sh` to make it executable.

KislistGUI.desktop (double-click launcher)

```
[Desktop Entry]
Type=Application
Name=KislistScraper GUI
Exec=gnome-terminal -- bash -c 'cd /path/to/KislistScraper_Pro_New && python3
-m scrapy'
```

```
Icon=utilities-terminal
Terminal=true
```

Edit `Exec` to the real path where you saved project. This is a simple launcher that opens a terminal.

README.md

```
# KislistScraper_Pro_New
```

Projekt Scrapy do pobierania zdjęć produktów z konkretnych sklepów (Domni, Łazienkarium, Dekordia) i zapisywania w ujednoliconej strukturze katalogów.

```
## Jak zainstalować
```

1. Załóż wirtualne środowisko:

```
```bash
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```
```

2. Uruchom przykładowy crawl:

```
```bash
scrapy crawl dekordia -a url="https://dekordia.pl/produkt/XXX"
scrapy crawl domni -a url="https://domni.pl/produkt/YYY"
scrapy crawl lazienkarium -a url="https://lazienkarium.pl/p/ZZZ"
```
```

3. Wyniki będą zapisywane w `./output/` (domyślnie). Sprawdź `settings.py` aby zmienić lokalizację.

```
## Co robi projekt
```

- Każdy spider potrafi wyciągnąć obrazy z galerii, `'srcset'`, `'picture'`/`'source'` i `'og:image'`.
- Pipeline pobiera obrazy HTTP-first (bez screenshotów).
- Jeżeli jest tylko jeden wariant, obrazy są zapisane bez podfolderu `'default'` (czyli pliki bez dodatkowego folderu wariantu).
- Dla Porty (jeżeli dodany) można dodać logikę kompozycji z klamkami (w tym szkielet jest przygotowane miejsce w pipeline).

```
## Dalsze kroki
```

- Możemy dodać Porty spider (JS-heavy) z Selenium jeśli potrzebujesz klikania i screenshotów.
- Możemy dodać integrację AI do wykrywania właściwych obrazów.

kislist_scraper/__init__.py

```
# package
```

kislist_scraper/items.py

```
from dataclasses import dataclass

@dataclass
class ProductItem:
    product_name: str
    product_url: str
    product_type: str
    variants: list # list of dicts: {name:str, images: [urls]}
```

kislist_scraper/settings.py

```
# Scrapy settings for kislist_scraper project
BOT_NAME = 'kislist_scraper'

SPIDER_MODULES = ['kislist_scraper.spiders']
NEWSPIDER_MODULE = 'kislist_scraper.spiders'

# Don't obey robots.txt (you can enable it if you prefer)
ROBOTSTXT_OBEY = False

# Configure item pipelines
ITEM_PIPELINES = {
    'kislist_scraper.pipelines.ImageSavePipeline': 300,
}

# Output folder (changeable)
OUTPUT_ROOT = 'output/Produkty_z_Kislist'

# Logging
LOG_LEVEL = 'INFO'
```

kislist_scraper/middlewares.py

```
# placeholder - can be extended (User-Agent rotation, proxies etc.)
from scrapy import signals
```

```
class KislistScraperMiddleware:
    @classmethod
    def from_crawler(cls, crawler):
        mw = cls()
        return mw
```

kislist_scraper/pipelines.py

```
import os
import re
import requests
from urllib.parse import urlparse
from pathlib import Path
from PIL import Image
from io import BytesIO

from scrapy.exceptions import DropItem
from .settings import OUTPUT_ROOT

class ImageSavePipeline:
    """Pipeline zapisujący zdjęcia w strukturze:
    OUTPUT_ROOT/<product_type>/<NNN_name>/<variant_name|(no variant)>/
    image_01.jpg

    Jeśli jest tylko 1 wariant i ma nazwę 'default' lub pusta, zapisujemy bez
    folderu wariantu.
    """

    def open_spider(self, spider):
        self.session = requests.Session()
        Path(OUTPUT_ROOT).mkdir(parents=True, exist_ok=True)

    def process_item(self, item, spider):
        # item: ProductItem dataclass or dict-like
        try:
            name = sanitize_filename(item.get('product_name') if
isinstance(item, dict) else item.product_name)
            ptype = item.get('product_type', 'produkty_inne') if
isinstance(item, dict) else item.product_type
            variants = item.get('variants') if isinstance(item, dict) else
item.variants

            # folder for product (per-type numbering is handled by spider)
            product_folder = Path(OUTPUT_ROOT) / ptype / name
            product_folder.mkdir(parents=True, exist_ok=True)
```

```

        # Normalize variants: list of dicts with 'name' and 'images'
        if not variants:
            raise DropItem('No variants found')

        # If single variant and name is default-ish – save directly in
product folder
        if len(variants) == 1:
            v = variants[0]
            vname = (v.get('name') or '').strip().lower()
            if vname in ('default', '', None):
                # save all images to product_folder as image_01.ext...
                self._download_images(v.get('images', []),
product_folder, prefix='image')
                return item

        # Otherwise save each variant into its own folder
        for v in variants:
            vname = sanitize_filename(v.get('name') or 'variant')
            variant_folder = product_folder / vname
            variant_folder.mkdir(parents=True, exist_ok=True)
            self._download_images(v.get('images', []), variant_folder,
prefix='image')

        return item
    except Exception as e:
        spider.logger.error(f"Pipeline error: {e}")
        raise DropItem(e)

def _download_images(self, urls, folder: Path, prefix='image'):
    idx = 1
    seen = set()
    for u in urls:
        if not u:
            continue
        if u in seen:
            continue
        seen.add(u)
        try:
            r = self.session.get(u, timeout=20)
            r.raise_for_status()
            # detect ext from content-type or URL
            ext = _ext_from_response(r, u)
            fname = folder / f"{prefix}_{idx:02d}{ext}"
            with open(fname, 'wb') as f:
                f.write(r.content)
            # optionally resize/validate image
            idx += 1
        except Exception:
            continue

```

```

def _ext_from_response(response, url):
    ctype = (response.headers.get('Content-Type') or '').lower()
    if 'jpeg' in ctype or 'jpg' in ctype:
        return '.jpg'
    if 'png' in ctype:
        return '.png'
    if 'webp' in ctype:
        return '.webp'
    # fallback from URL
    path = urlparse(url).path
    if '.' in path:
        return '.' + path.split('.')[-1]
    return '.jpg'

def sanitize_filename(name: str) -> str:
    if not name:
        return 'unknown'
    replacements = {
        'à': 'a', 'á': 'c', 'ä': 'e', 'ł': 'l', 'ń': 'n',
        'ó': 'o', 'ś': 's', 'ż': 'z', 'ź': 'z',
        'Ą': 'A', 'Ć': 'C', 'Ę': 'E', 'Ł': 'L', 'Ń': 'N',
        'Ó': 'O', 'Ś': 'S', 'Ż': 'Z', 'Ź': 'Z'
    }
    for pl, en in replacements.items():
        name = name.replace(pl, en)
    name = re.sub(r'[:<>:"/\|?*]', '_', name)
    name = re.sub(r'\s+', '_', name)
    name = name.strip('_')
    return name[:120]

```

kislist_scraper/spiders/__init__.py

```
# spiders package
```

kislist_scraper/spiders/base_spider.py

```

import scrapy
from ..items import ProductItem
from urllib.parse import urlparse
from bs4 import BeautifulSoup
import re

class BaseProductSpider(scrapy.Spider):

```

```
"""Base spider with helpers: extract images from <img>, <picture>, srcset and
og:image.
```

```
Subclasses must implement parse_product(self, response) -> dict with
product data
```

```
"""
```

```
def extract_images_from_dom(self, response):
```

```
"""Return list of absolute image URLs found in page (prioritize large
images)."""
```

```
    urls = []
```

```
    soup = BeautifulSoup(response.text, 'lxml')
```

```
    # 1) <picture> and <source srcset>
```

```
    for pic in soup.select('picture'):
```

```
        # prefer source@srcset largest
```

```
        sources = pic.find_all('source')
```

```
        for s in sources:
```

```
            srcset = s.get('srcset') or s.get('data-srcset')
```

```
            if srcset:
```

```
                candidate = self._pick_largest_from_srcset(srcset)
```

```
                if candidate:
```

```
                    urls.append(self._abs(candidate, response))
```

```
    img = pic.find('img')
```

```
    if img and img.get('src'):
```

```
        urls.append(self._abs(img.get('src'), response))
```

```
    # 2) <img src/srcset/data-src>
```

```
    for img in soup.find_all('img'):
```

```
        for attr in ('data-src', 'data-original', 'src', 'data-lazy',
'data-srcset'):
```

```
            val = img.get(attr)
```

```
            if not val:
```

```
                continue
```

```
            if attr in ('srcset', 'data-srcset', 'data-src') or ',' in
```

```
val:
```

```
                # srcset-like
```

```
                candidate = self._pick_largest_from_srcset(val)
```

```
                if candidate:
```

```
                    urls.append(self._abs(candidate, response))
```

```
            else:
```

```
                urls.append(self._abs(val, response))
```

```
    # 3) og:image fallback
```

```
    og = soup.find('meta', property='og:image')
```

```
    if og and og.get('content'):
```

```
        urls.append(self._abs(og.get('content'), response))
```

```
    # filter/normalize
```

```
    out = []
```

```

seen = set()
for u in urls:
    u = u.split('?')[0]
    if u and u not in seen and self._looks_like_product_image(u):
        seen.add(u)
        out.append(u)
return out

def _pick_largest_from_srcset(self, srcset: str):
    parts = [p.strip() for p in srcset.split(',') if p.strip()]
    best = None
    best_w = 0
    for p in parts:
        if ' ' in p:
            url, w = p.rsplit(' ', 1)
            try:
                if w.endswith('w'):
                    wi = int(w[:-1])
                    if wi > best_w:
                        best_w = wi
                        best = url
            except Exception:
                continue
        else:
            best = p
    return best

def _abs(self, src, response):
    return response.urljoin(src)

def _looks_like_product_image(self, url: str):
    url_l = url.lower()
    # ignore icons, logos, social, pixel, sprite
    bad = ['logo', 'icon', 'sprite', 'banner', 'thumb', 'pixel',
'placeholder']
    if any(b in url_l for b in bad):
        return False
    # ensure has image extension
    return any(ext in url_l for ext in ('.jpg', '.jpeg', '.png',
'.webp'))

```

kislist_scraper/spiders/domni_spider.py

```

import scrapy
from .base_spider import BaseProductSpider
from ..items import ProductItem

class DomniSpider(BaseProductSpider):

```

```

name = 'domni'
allowed_domains = ['domni.pl', 'media.domni.pl']

def start_requests(self):
    url = getattr(self, 'url', None)
    if not url:
        raise RuntimeError('domni spider requires -a url=...')
    yield scrapy.Request(url, callback=self.parse)

def parse(self, response):
    # product name
    title = response.css('h1::text').get() or response.css('.product-
title::text').get() or response.xpath('//title/text()').get()
    images = self.extract_images_from_dom(response)

    # Domni often has swiper with big images; our Base extractor handles
    <picture> and srcset

    variants = []
    # Try to detect per-image alt/title containing color words
    for img in images:
        variants.append({
            'name': 'default',
            'images': images
        })
        break

    item = {
        'product_name': title.strip() if title else 'domni_product',
        'product_url': response.url,
        'product_type': 'plytki_ceramiczne',
        'variants': variants
    }
    yield item

```

kislist_scraper/spiders/lazienkarium_spider.py

```

import scrapy
from .base_spider import BaseProductSpider

class LazienkariumSpider(BaseProductSpider):
    name = 'lazienkarium'
    allowed_domains = ['lazienkarium.pl', 'cdn.lazienkarium.pl']

    def start_requests(self):
        url = getattr(self, 'url', None)
        if not url:
            raise RuntimeError('lazienkarium spider requires -a url=...')

```

```

        yield scrapy.Request(url, callback=self.parse)

    def parse(self, response):
        title = response.css('h1::text').get() or response.xpath('//title/text()').get()

        # Lazienkarium gallery uses lg-item and thumbnails – Base extractor should capture them
        images = self.extract_images_from_dom(response)

        # Also try to extract from .lg-item img src attributes specifically
        thumbs = response.xpath("//div[contains(@class,'lg-item')]/img/@src").getall()
        for t in thumbs:
            u = response.urljoin(t)
            if u not in images:
                images.append(u)

        variants = [{'name': 'default', 'images': images}]

        item = {
            'product_name': title.strip() if title else 'lazienkarium_product',
            'product_url': response.url,
            'product_type': 'bateria_umywalkowa',
            'variants': variants
        }
        yield item

```

kislist_scraper/spiders/dekordia_spider.py

```

import scrapy
from .base_spider import BaseProductSpider

class DekordiaSpider(BaseProductSpider):
    name = 'dekordia'
    allowed_domains = ['dekordia.pl', 'storage.dekordia.pl']

    def start_requests(self):
        url = getattr(self, 'url', None)
        if not url:
            raise RuntimeError('dekordia spider requires -a url=...')
        yield scrapy.Request(url, callback=self.parse)

    def parse(self, response):
        title = response.css('h1::text').get() or response.xpath('//title/text()').get()

```

```

# Dekordia often uses <picture> and glightbox anchors – extract them
images = self.extract_images_from_dom(response)

# also get links from a.glightbox href (these are often big images)
glinks = response.xpath("//a[contains(@class,'gllightbox')]/
@href").getall()
for g in glinks:
    u = response.urljoin(g)
    if u not in images:
        images.append(u)

variants = [{'name': 'default', 'images': images}]

item = {
    'product_name': title.strip() if title else 'dekordia_product',
    'product_url': response.url,
    'product_type': 'listwy_przypodlogowe',
    'variants': variants
}
yield item

```

Co jeszcze zrobimy dalej (po wdrożeniu)

1. Dodać spidera dla porta.com.pl (JS-heavy) — z Selenium integration lub scrapy-splash/Playwright. Ten krok dopracuje kompozyty klamek.
2. Dodać lepszą detekcję wariantów nazwy (mapowanie kolorów) jak w Twoim `variant_detector_simple.py`.
3. Dodać deduplikację i heurystyki filtrowania, oraz zapis JSON `variant_info.json` per wariant.
4. Opcjonalnie GUI front-end (Tkinter) lub prosty wrapper shell script / .desktop (już jest szkic).

Jeżeli chcesz, mogę teraz: - wygenerować pliki w tym katalogu na Twoim dysku (zapiszę je w /mnt/RZECZY/KislistScraper_Pro_New), - albo najpierw wprowadzić dodatkowe zmiany: integrację z Pillow do kompozycji, dokładniejszą logikę "no-default-folder", mapowanie kolorów, lub Porta spider z Selenium.

Powiedz co chcesz zrobić dalej — i czy mam zapisać te pliki na dysku teraz (i gdzie dokładnie).